

ANÁLISE EXPERIMENTAL ESTRUTURADA: GENRESEARCH-SERVIDOR

Docente: Glauco de Figueiredo Carneiro

Discentes: Beathriz Laurent, Samyr Teles, Rafael de Jesus, Luan Feitosa, Matheus de Oliveira, Evelyn Hilthegarden

Introdução & Problema

Gargalos de SO em Cargas Intensivas Emuladas (RAG)



Escopo do GenResearch

Ambiente simulado para investigar conceitos fundamentais de SO: gerenciamento de processos, escalonamento, memória virtual, paginação e operações de E/S sob cargas no estilo Retrieval-Augmented Generation (RAG).



O Problema Investigado

Identificação e mitigação de gargalos de E/S, latência e paginação no processamento de grandes bases de dados, comparando leitura tradicional vs mmap, escalabilidade de filas concorrentes e riscos de corrida em cache.

EIXOS SISTEMÁTICOS DA INVESTIGAÇÃO EXPERIMENTAL

E/S E MEMÓRIA VIRTUAL

Impacto do volume de dados em RAM, paginação, faltas de página e comparação entre leitura read() vs mmap.

CACHE E CONCORRÊNCIA

Taxa de acerto do cache LRU, redução de latência e ganhos de vazão com múltiplos trabalhadores na fila de requisições.

SINCRONIZAÇÃO & CORRIDA

Condições de corrida em recursos compartilhados, mitigações por exclusão mútua (Mutex) e trade-offs de tempo de espera.

APLICAÇÕES RAG E LIMITAÇÕES

Relação com embeddings/LLMs e barreiras na generalização para GPUs dedicadas ou sistemas distribuídos.

Design do Servidor e Emulação de Carga Computacional

Arquitetura Produtor-Consumidor

Servidor HTTP Python 3.12 com threads produtoras (requisições web) e trabalhadoras (consumidoras da fila limite 8).

Armazenamento persistente SQLite para índices e Cache LRU em memória volátil (8 posições) para resultados recentes.

Emulação por PBKDF2

Embeddings e respostas simuladas através do algoritmo criptográfico PBKDF2, sem uso de redes neurais ativas (ex: Ollama).

Gera alto custo sintético de CPU por requisição, emulando perfeitamente a latência e o peso de inferências reais.

5. RESULTADOS — EXPERIMENTO C5

Tabela 1: Variação Controlada do Corpus (256 KiB vs 1.024 KiB)

Corpus KiB	Rep.	Índice B	Trechos	Ingestão ms	Lote ms	RSS KiB	VSZ KiB	minflt
256	1	716.800	660	49,902	95,160	30.240	334.888	2.601
256	2	716.800	660	49,803	96,560	29.784	334.888	2.382
256	3	716.800	660	49,788	97,971	29.788	334.888	2.388
1024	1	2.797.568	2.639	**200,981**	359,459	38.836	412.652	6.226
1024	2	2.797.568	2.639	**197,846**	390,939	37.944	412.652	5.985
1024	3	2.797.568	2.639	**188,051**	362,826	38.996	339.944	6.274

Achado Principal: Quatro vezes mais dados elevaram o tempo de ingestão de ~49 ms para ~200 ms. O RSS subiu de ~29 MiB para ~38 MiB sem provocar paginação severa ou swap.

Tabela 2: Leitura Convencional (C1) vs Mapeamento mmap (C2)

Modo	MiB	Rep.	Leitura ms	RSS pico KiB	RSS fim KiB	VSZ KiB	minflt	read_bytes
C1 (read)	2	1	2,076	40.672	25.388	37.516	516	0
C2 (mmap)	2	1	11,217	40.672	25.296	37.516	548	2.158.592
C1 (read)	**16**	1	**30,033**	40.672	26.336	38.596	1.028	14.618.624
C2 (mmap)	**16**	1	**10,356**	42.480	26.432	38.596	1.145	**0**

Vantagem do mmap: Em arquivos de 16 MiB, o `mmap` reduziu o tempo de abertura de 30,0 ms para 10,3 ms, evitando cópias manuais de disco via `read_bytes` do user space.

7. RESULTADOS — EXPERIMENTOS C4 & C3

Impacto do Cache LRU e Escalabilidade Multithreading

Tabela 3: Cache (C4)

Cache	Padrão	Hits/Miss	Lat. ms	Req/s
0	Rep.	0 / 12	2,540	380,4
0	Dist.	0 / 12	2,589	375,9
8	Rep.	**11 / 1 (91,7%)**	**0,918**	**1015,9**
8	Dist.	0 / 12	2,653	367,4

Com acertos de cache (91,67%), a latência despencou de 2,6 ms para 0,9 ms, triplicando a vazão.

Tabela 4: Concorrência (C3)

Trab.	Vazão req/s	Lat. ms	p95 ms	Espera ms
1 worker	436,2	8,620	9,244	5,647
2 workers	730,4	5,161	6,189	1,651
4 workers	**837,4**	**4,537**	7,016	**0,111**

Aumentar para 4 workers reduziu o tempo de espera na fila de 5,6 ms para quase zero (0,11 ms).

8. RESULTADOS — PARTE D

Tabela 5: Sincronização do Mutex e Eliminação do Retrabalho

Modo	Rep.	Trab.	Gerações	Duplicadas	Hits	Misses	Lat. méd. ms	Espera mutex ms
D1 (Sem Lock)	1	4	4	3	0	4	32,785	0,000
D2 (Com Mutex)	1	4	1	0	3	1	32,865	23,982

Trade-off Demonstrado: O Mutex em D2 eliminou completamente as 3 gerações duplicadas (trabalho redundante) do cenário D1, garantindo 3 hits. No entanto, introduziu ~24 ms de tempo de espera/bloqueio para as threads.

Comportamento de Memória, Paginação e E/S

Cópia para User Space

Leituras convencionais via read() exigem duplicação das páginas do Page Cache do kernel para o espaço do usuário. Em arquivos grandes (16 MiB), isso introduz latências substanciais no arranque frio.

Overhead de Cópia

Faltas de Página e Swap

O mmap eleva levemente as faltas de página suaves (minflt) para ligar a memória virtual sob demanda. Em todos os cenários, faltas maiores (majflt) foram 0, sem atividade de Swap ou thrashing.

Sem Thrashing / Swap

0

APLICAÇÕES PRÁTICAS NA INFRAESTRUTURA DE LLMS

Modelos e Weights



Frameworks como PyTorch e Ollama usam mmap para carregar pesos e vetores densos sem copiar gigabytes para o user space.

Mmap em LLMs

Context Caches



O reprocessamento sem cache evidencia o custo gigantesco de tensores. Caches atômicos evitam recálculos caros em prompts frequentes.

Economia de Cálculo

Pipeline RAG



Demonstra a necessidade de otimizar a camada de armazenamento de embeddings e índice vetorial para reduzir a latência total do usuário.

Redução de Latência

Fatores que Impedem a Generalização Direta

Simulação Criptográfica (PBKDF2)

A carga de CPU foi simulada via PBKDF2 em vez de multiplicações matriciais de tensores reais. As GPUs dedicadas (RTX 2050/3050) não foram acionadas para processamento de inferência.

Carga Sintética

Ambiente Monolítico Local

Sistemas distribuídos reais em nuvem enfrentam gargalos de rede (Ethernet/InfiniBand) e sincronização entre nós que diferem do Mutex local em memória compartilhada no WSL2.

Escopo de Processo Único

CONCLUSÃO DA AVALIAÇÃO EXPERIMENTAL (AV2)

- Eficiência e Processamento: Os testes comprovaram que o uso de mapeamentos virtuais contínuos sob demanda (voltados a operações de E/S) otimiza o carregamento computacional e garante a consistência dos blocos processados de forma assíncrona. Isso elimina cópias desnecessárias e aproveita o fluxo de paginação suave do Linux.
- Escalabilidade e o Trade-off de Sistemas Operacionais: Embora aumentar o número de workers paralelos resolva os tempos de espera em filas web, surge o desafio das transações cruzadas em espaço compartilhado. Para evitar que sistemas pseudo-RAG sofram com repetições custosas do modelo (utilizando exclusão atômica em memória), foi necessário introduzir um retardo intencional. Esse cenário reflete o clássico trade-off dos Sistemas Operacionais: equilibrar a coordenação segura de processos em um ambiente de tempo e recursos computacionais finitos.

ENCERRAMENTO

Obrigado!



Agradecimentos